GamingCPU Virtual Platform

Architecture Specification Document

SystemC/TLM-2.0 Behavioral Model of a Custom RV32IMAC SoC

Revision 1.0, March 5, 2026

Rafeed Khan
Internal Design Document

Abstract

This document provides the complete architectural specification for the GamingCPU-VP, a SystemC/TLM-2.0 transaction-level behavioral model of the GamingCPU-SoC custom RISC-V system-on-chip for IEEE UCF. The IEEE UCF GamingCPU-SoC is an RV32IMAC-based SoC designed to boot and run the classic first-person shooter DOOM at interactive frame rates on an Artix-7 FPGA. The GamingCPU-VP replicates the full memory map, register interfaces, interrupt architecture, and peripheral behavior of the RTL implementation in pure behavioral C++, enabling (a) early software development before RTL completion, (b) golden-reference validation of the RTL against a known-correct behavioral model, and (c) rapid architectural exploration of the memory hierarchy, DMA engine, and video/audio pipelines. The document covers all subsystems, every source file, the complete memory map, detailed transaction flow scenarios, DOOM integration analysis, verification strategy, build procedures, and failure mode analysis. It is intended for review by hardware architecture faculty and for use as the canonical internal design reference.

Contents

1 Executive Summary **3**
1.1 Purpose of the Virtual Platform 3
1.2 Relationship to RTL Implementation . . 3
1.3 Design Philosophy 4

2 Complete Architectural Overview **4**
2.1 CPU Subsystem: Instruction Set Simulator 4
2.1.1 ISS Design 4
2.1.2 Threading Model 4
2.1.3 Decode/Execute Separation . . 4
2.1.4 CSR Handling 5
2.1.5 MMU Behavioral Model 5
2.2 Bus Architecture 5
2.2.1 TLM-2.0 Routing Bus 5
2.2.2 DMI Usage 5
2.2.3 Address Decode Strategy 5
2.3 Memory Hierarchy 5
2.3.1 RAM Model 6
2.3.2 BootROM Model 6
2.3.3 SD Card Image Backing 6
2.4 Interrupt Architecture 6
2.4.1 CLINT 6
2.4.2 PLIC 6
2.4.3 Trap Handling Path 6
2.5 Peripheral Abstraction Philosophy . . . 6
2.6 Video Pipeline 6
2.7 Audio Pipeline 7
2.8 DMA Engine 7
2.9 Debug Architecture 7

3 Detailed Directory Walkthrough **7**
3.1 `deps/` 7
3.1.1 `deps/setup.sh` 7
3.2 `src/main.cpp` 7
3.3 `src/platform/` 8
3.3.1 `src/platform/gamingcpu_vp.h` and `gamingcpu_vp.cpp` 8
3.3.2 `src/platform/platform_config.h` 8
3.4 `src/cpu/` 8
3.4.1 `src/cpu/iss.h` and `iss.cpp` 8
3.4.2 `src/cpu/decode.h` and `decode.cpp` 9
3.4.3 `src/cpu/execute.h` and `execute.cpp` 9

3.4.4 src/cpu/rv32m.h and rv32m.cpp 9
3.4.5 src/cpu/rv32a.h and rv32a.cpp 9
3.4.6 src/cpu/csr.h and csr.cpp 9
3.4.7 src/cpu/mmu.h and mmu.cpp 10
3.4.8 src/cpu/trap.h and trap.cpp 10
3.4.9 src/cpu/rv32_defs.h . . 10
3.5 src/bus/ 10
3.5.1 src/bus/tlm_bus.h and tlm_bus.cpp 10
3.6 src/mem/ 10
3.6.1 src/mem/memory.h and memory.cpp 10
3.6.2 src/mem/bootrom.h and bootrom.cpp 10
3.7 src/irq/ 10
3.7.1 src/irq/clint.h and clint.cpp 10
3.7.2 src/irq/plic.h and plic.cpp 11
3.8 src/io/ 11
3.8.1 src/io/uart.h and uart.cpp 11
3.8.2 src/io/gpio.h and gpio.cpp 11
3.8.3 src/io/timer.h and timer.cpp 11
3.8.4 src/io/spi.h and spi.cpp 11
3.9 src/sd/ 11
3.9.1 src/sd/sd_ctrl.h and sd_ctrl.cpp 11
3.9.2 src/sd/sd_card_-model.h and sd_card_-model.cpp 11
3.10 src/video/ 11
3.10.1 src/video/fb_ctrl.h and fb_ctrl.cpp 12
3.10.2 src/video/palette.h and palette.cpp 12
3.11 src/audio/ 12
3.11.1 src/audio/audio_out.h and audio_out.cpp 12
3.12 src/dma/ 12
3.12.1 src/dma/dma_engine.h and dma_engine.cpp 12
3.13 src/debug/ 12
3.13.1 src/debug/gdb_-server.h and gdb_-server.cpp 12
3.14 src/util/ 12
3.14.1 src/util/elf_loader.h and elf_loader.cpp 12
3.14.2 src/util/logging.h and logging.cpp 13
3.15 sw/ 13

4 Memory Map Specification 13
4.1 Interrupt Sources 13
4.2 Framebuffer Layout 13
4.3 Audio Ring Buffer Layout 13

5 Transaction Flow Scenarios 13
5.1 CPU Instruction Fetch 13
5.2 MMU Page Walk 14
5.3 UART Transmit 14
5.4 Timer Interrupt 14
5.5 DMA Framebuffer Transfer 14
5.6 DOOM Frame Render Path 14

6 DOOM Integration Analysis 14
6.1 Why DOOM Works on This Architecture 14
6.2 Required ISA Extensions 14
6.3 Fixed-Point Math Considerations 15
6.4 Framebuffer Constraints 15
6.5 Audio Timing 15
6.6 SD Card Access Patterns 15
6.7 Expected Performance Bottlenecks . . . 15

7 Verification & Testing Strategy 15
7.1 Unit Testing Philosophy 15
7.2 TLM-Level Assertions 15
7.3 Transaction Tracing Strategy 15
7.4 Deterministic Replay 15
7.5 ISA Compliance Flow 16
7.6 Memory Corruption Detection 16
7.7 Interrupt Stress Tests 16
7.8 Bus Contention Tests 16
7.9 SDL Rendering Validation 16
7.10 Golden Frame Checksum Strategy . . . 16
7.11 Cross-Validation Against RTL 16
7.12 Continuous Integration Structure 16

8 Build & Execution Instructions 17
8.1 Dependency Setup 17
8.2 CMake Build Flow 17
8.3 ELF Loading 17
8.4 Running hello_uart 17

8.5 Running DOOM 17
8.6 Attaching riscv-gdb 17
8.7 Debugging Checklist 17

9 Failure Mode Analysis 17
9.1 TLM Deadlocks 17
9.2 DMI Invalidation Issues 18
9.3 Incorrect MMU Translation 18
9.4 Interrupt Starvation 18
9.5 Frame Tearing 18
9.6 Audio Underruns 18
9.7 SD Corruption Scenarios 18

10 Future Work 18
10.1 Cycle-Accurate Refinement 18
10.2 Hybrid Co-Simulation with RTL 18
10.3 Formal Verification Hooks 18
10.4 Trace Export for Waveform Comparison 19

11 Conclusion 19

1 Executive Summary

1.1 Purpose of the Virtual Platform

The GamingCPU-VP exists to solve three problems that arise in custom SoC development.

First, the RTL implementation of the GamingCPU-SoC requires months of effort across the CPU pipeline, cache hierarchy, AXI fabric, and peripheral controllers. During this period, the firmware and application software teams, particularly the DOOM porting team, cannot begin integration testing because no executable hardware model exists. The GamingCPU-VP provides a bit-accurate, register-compatible execution environment that runs the *identical* cross-compiled ELF binaries that will later execute on FPGA silicon. Every memory-mapped register, every interrupt source, and every DMA channel is present at the same address with the same semantics. The software team observes no difference between the VP and the eventual FPGA, aside from wall-clock execution speed.

Second, the VP serves as the *golden reference model* for RTL verification. When both the VP and the Verilator-simulated RTL execute the same firmware binary, their instruction traces—program counter, decoded instruction, register file writes, and memory transactions—must match cycle-for-cycle at retirement boundaries. Any divergence constitutes a bug in either the VP or the RTL, and because the VP is dramatically simpler (no pipeline hazards, no cache coherence FSMs, no clock-domain crossings), the VP is presumed correct unless proven otherwise. This asymmetry in complexity is the fundamental argument for maintaining a behavioral golden model alongside an RTL implementation.

Third, the VP enables rapid architectural exploration. Questions such as "What happens to DOOM frame rate if we double the DMA burst length?" or "Does the audio ring buffer underrun if we reduce it from 4 KB to 2 KB?" can be answered in seconds on the VP, whereas the same experiment on RTL simulation would require hours.

1.2 Relationship to RTL Implementation

Table 1 summarizes the correspondence between RTL modules and VP modules. The mapping is one-to-one at the peripheral level: every SystemVerilog module in the `rtl/` tree has a corresponding C++ class in the `src/` tree that implements the same register interface and behavioral semantics. The key differences are:

- The 5-stage pipeline (`fetch.sv` through `writeback.sv`) is replaced by a single-threaded fetch-decode-execute loop in `iss.cpp`.
- The cache hierarchy (`icache.sv`, `dcache.sv`) is replaced by TLM-2.0 Direct Memory Interface (DMI) pointers.
- The AXI crossbar (`axi_crossbar.sv`) is replaced by a TLM address-routing bus (`tlm_bus.cpp`).
- Clock-domain crossings, reset synchronizers, and PLL configuration are absent entirely.

Table 1: RTL to VP Module Correspondence

RTL Module	VP Equivalent
`rv32_core.sv` (5-stage)	`iss.cpp` (behavioral loop)
`icache.sv`, `dcache.sv`	TLM DMI (zero-copy pointer)
`axi_crossbar.sv`	`tlm_bus.cpp` (address router)
`sv32_mmu.sv` + `tlb.sv` + `ptw.sv`	`mmu.cpp` (software page walk)
`clint.sv`	`clint.cpp`
`plic.sv`	`plic.cpp`
`uart16550.sv`	`uart.cpp`
`fb_ctrl.sv` + `vga_timing.sv`	`fb_ctrl.cpp` (SDL2 output)
`i2s_tx.sv` + `audio_dma.sv`	`audio_out.cpp` (SDL2 audio)
`sd_spi_ctrl.sv`	`sd_ctrl.cpp` (host file backing)
`axi_dma.sv`	`dma_engine.cpp`
`riscv_dm.sv` + `jtag_tap.sv`	`gdb_server.cpp` (TCP RSP)
`bootrom.sv`	`bootrom.cpp` (read-only memory)
`sram_dualport.sv` + DDR	`memory.cpp` (flat array)

1.3 Design Philosophy

The GamingCPU-VP is a *loosely-timed* (LT) model in TLM-2.0 terminology. It prioritizes functional correctness and simulation speed over cycle-accurate timing. Specific design decisions:

Behavioral accuracy over cycle accuracy. The ISS executes one instruction per invocation of its `run()` method. It does not model pipeline stages, forwarding paths, or stall cycles. Each instruction consumes a configurable quantum of simulated time (default: 1 cycle at the nominal clock frequency). This means the VP cannot predict exact cycle counts for a given code sequence, but it *can* predict functional behavior: register values, memory contents, interrupt delivery order, and peripheral state transitions.

Temporal decoupling. The ISS runs ahead of the SystemC kernel by a configurable *local quantum* (default: 100 μs of simulated time). This reduces context switches to the SystemC scheduler from once-per-instruction to once-per-quantum, yielding a 10-100$\times$ simulation speedup. The tradeoff is reduced timing accuracy for interrupt latency; interrupts are only observed at quantum boundaries.

DMI for memory. All instruction fetches and the majority of load/store operations bypass the TLM socket path entirely via DMI pointers obtained during initialization. This converts memory accesses from function calls through the SystemC elaboration hierarchy into direct C++ pointer dereferences, which is the single largest performance optimization in the VP.

2 Complete Architectural Overview

2.1 CPU Subsystem: Instruction Set Simulator

2.1.1 ISS Design

The ISS is implemented as a single `sc_module` named `ISS` with one TLM-2.0 initiator socket for memory access. Internally, it maintains the complete RV32IMAC architectural state:

- 32 general-purpose registers (`x0-x31`), where `x0` is hardwired to zero.
- The program counter (`pc`).
- The full set of M-mode and S-mode CSRs: `mstatus`, `mie`, `mip`, `mtvec`, `mepc`, `mcause`, `mtval`, `mscratch`, `medeleg`, `mideleg`, `satp`, `stvec`, `sepc`, `scause`, `stval`, `sscratch`, and performance counters `mcycle`, `minstret`.
- A privilege level register tracking the current mode (M/S/U).
- An LR/SC reservation set address and valid flag for the A extension.

2.1.2 Threading Model

The ISS registers a single `SC_THREAD` named `run`. This thread executes an infinite loop:

```
void ISS::run() {
  while (true) {
    // 1. Check pending interrupts
    if (check_pending_interrupts())
      take_trap(pending_interrupt_cause);
    // 2. Fetch instruction
    uint32_t instr = mem_read32(pc);
    // 3. Decode
    DecodedInstr d = decode(instr);
    // 4. Execute
    execute(d);
    // 5. Advance PC
    pc = next_pc;
    // 6. Increment counters
    csr.minstret++;
    csr.mcycle++;
    // 7. Advance local time
    local_time += CYCLE_TIME;
    if (local_time > quantum)
      sync_and_yield();
  }
}
```

Listing 1: ISS main loop (simplified)

The `sync_and_yield()` call invokes `tlm::tlm_quantumkeeper::sync()` which calls `sc_core::wait(local_time)`, yielding control to the SystemC kernel. This allows peripheral `SC_THREAD`s (e.g., the CLINT timer comparator) to execute.

2.1.3 Decode/Execute Separation

Instruction decoding is implemented in `decode.cpp` as a combinational function (no state, no side effects). It accepts a raw 32-bit instruction word and returns a `DecodedInstr` structure containing the opcode class, source/destination register indices, immediate value, and function codes. Compressed (C extension) instructions are detected by examining bits [1:0]; if they are not `2'b11`, the instruction is 16 bits wide and is expanded to its 32-bit equivalent before further decoding.

Execution is implemented in `execute.cpp` as a large `switch` statement over the opcode class. Each case reads source registers from the register file, performs the operation (ALU, branch comparison, memory access, CSR read/write), writes the destination register, and computes `next_pc`. The M extension (`rv32m.cpp`) handles `MUL`, `MULH`, `MULHSU`, `MULHU`, `DIV`, `DIVU`, `REM`, and `REMU` as straightforward 64-bit C++ inte-

ger operations cast to 32-bit results. The A extension (`rv32a.cpp`) implements `LR.W` and `SC.W` using the reservation set: `LR.W` records the aligned address and sets a valid flag; `SC.W` checks the flag and address, conditionally writes, and clears the reservation. Any intervening store to the reservation address by any master (including DMA) must invalidate the reservation.

2.1.4 CSR Handling

CSR accesses are mediated by the `csr.cpp` module, which implements `read_csr(addr)` and `write_csr(addr, val)` methods. Each CSR address is mapped to a field in the CSR structure. Read-only CSRs (`mvendorid`, `marchid`, `mimpid`, `mhartid`) return constant values. WARL (Write Any, Read Legal) fields such as `mstatus`.MPP are masked on write to enforce legal values. The `mip` register is partially read-only: the MTIP, MSIP, and MEIP bits are driven by the CLINT and PLIC respectively and cannot be written by software. The `satp` register triggers a TLB flush (in the behavioral MMU model, this clears the software TLB cache) on every write.

2.1.5 MMU Behavioral Model

The Sv32 MMU is implemented in `mmu.cpp` as a software page table walker. When the ISS performs a memory access and `satp`.MODE is set to Sv32, the virtual address is translated as follows:

1. Extract VPN[1] (bits [31:22]) and VPN[0] (bits [21:12]).
2. Read the first-level PTE from physical address `satp.PPN * 4096 + VPN[1] * 4`.
3. If the PTE is a leaf (R, W, or X bits set), compute the physical address from the PTE's PPN and the page offset. This is a 4 MB superpage.
4. If the PTE is a pointer (R=W=X=0), read the second-level PTE from `PTE.PPN * 4096 + VPN[0] * 4`.
5. Check permission bits (U, R, W, X) against the current privilege level and access type. Raise a page fault exception if permissions are violated.
6. Check and set the A (accessed) and D (dirty) bits as required by the RISC-V specification.

A software TLB with 64 entries (configurable) caches recent translations. The TLB is fully associative with random replacement. It is flushed on `SFENCE.VMA` execution or `satp` write. The TLB is purely a performance optimization for the VP; it does not model the RTL TLB's specific associativity or replacement policy.

PTE fetches during the page walk are performed via the same TLM initiator socket as normal memory accesses, ensuring that page table memory is read from the same backing store as application data.

2.2 Bus Architecture

2.2.1 TLM-2.0 Routing Bus

The `tlm_bus.cpp` module replaces the AXI crossbar from the RTL. It is a `sc_module` with one TLM-2.0 target socket (receiving transactions from the CPU and DMA engine) and N initiator sockets (one per slave peripheral). Address decoding is performed by a sorted table of (base, size, target_index) tuples configured during elaboration.

When `b_transport` is called on the target socket, the bus extracts the transaction address, performs a binary search over the address map, and forwards the transaction to the corresponding initiator socket with the address adjusted to a peripheral-local offset. If no address range matches, the bus sets the TLM response status to `TLM_ADDRESS_ERROR_RESPONSE` and returns.

2.2.2 DMI Usage

During the first memory access to the RAM and BootROM regions, the bus requests a DMI pointer from the target via `get_direct_mem_ptr`. The memory model returns a pointer to its internal `std::vector` backing store along with the valid address range. The bus caches this DMI descriptor and returns it to the ISS. Subsequent accesses within the DMI-valid range bypass the socket path entirely.

DMI is *not* granted for MMIO regions (peripherals). Any access to an address in the MMIO range always traverses the full TLM socket path, ensuring that peripheral side effects (register writes, interrupt triggers) are correctly modeled.

2.2.3 Address Decode Strategy

The address map is defined in `platform_config.h`, generated from `specs/memory_map.yaml`. The bus constructor iterates over this map and binds each address range to the corresponding target socket. The decode is a simple linear scan (acceptable for <20 peripherals) or, optionally, a binary search for larger configurations.

2.3 Memory Hierarchy

2.3.1 RAM Model

The RAM is modeled as a contiguous `std::vector<uint8_t>` sized to the DDR3 capacity (128 MB for the Arty A7 configuration). It supports both TLM blocking transport and DMI. There is no distinction between SRAM and DDR in the VP; both are merged into a single flat memory model. This is a deliberate simplification: the VP does not model DDR latency, refresh, or calibration. The MIG controller and its calibration FSM are entirely absent.

2.3.2 BootROM Model

The BootROM is a read-only memory initialized from a binary file at elaboration time. Writes are silently ignored (with an `SC_WARNING`). The BootROM contents correspond exactly to the output of the `sw/bootrom/Makefile` build.

2.3.3 SD Card Image Backing

The SD controller model (`sd_ctrl.cpp`) does not emulate the SPI or SDIO wire protocol. Instead, it accepts register writes that specify a block address and a destination DMA address, opens a host-side file (the SD card image, typically `images/doom1.img`), seeks to the requested 512-byte block, reads it into an internal buffer, and performs a TLM write transaction to copy the block into VP memory. From the firmware's perspective, the register interface is identical to the RTL SD controller.

2.4 Interrupt Architecture

2.4.1 CLINT

The Core Local Interruptor implements three registers: `mtime` (64-bit free-running counter), `mtimecmp` (64-bit comparator), and `msip` (software interrupt pending). An internal `SC_THREAD` increments `mtime` at the configured tick rate. When `mtime >= mtimecmp`, the CLINT asserts the *timer_irq* output, which is wired to the ISS's `mip.MTIP` input. Writing a new value to `mtimecmp` clears the interrupt.

2.4.2 PLIC

The Platform Level Interrupt Controller collects external interrupt sources from all peripherals (UART, timer, GPIO, SD, video vsync, audio underrun). Each source has a configurable priority (0-7). The PLIC exposes a *claim* register: when the ISS reads the claim register, it receives the ID of the highest-priority pending interrupt and the interrupt is marked as in-service. Writing the ID to the *complete* register marks the interrupt as finished. The PLIC asserts *external_irq* to the ISS's `mip.MEIP` whenever any enabled interrupt is pending and its priority exceeds the threshold.

2.4.3 Trap Handling Path

When the ISS detects a pending interrupt (at the top of the fetch loop), it performs the standard RISC-V trap entry sequence: save `pc` to `mepc` (or `sepc` for delegated interrupts), set `mcause` with the interrupt bit and cause code, set `mstatus`.MPIE to `mstatus`.MIE, clear `mstatus`.MIE, and jump to `mtvec`. For vectored mode (`mtvec`[1:0] = 1), the jump target is `mtvec.BASE + 4 * cause`.

2.5 Peripheral Abstraction Philosophy

Every peripheral in the VP implements the same register map as its RTL counterpart, as defined in the `specs/registers/*.yaml` files. The internal implementation, however, models only the *observable behavior*, the values returned by register reads and the side effects triggered by register writes, not the cycle-by-cycle microarchitecture.

For example, the RTL UART (`uart16550.sv`) contains RX/TX FIFOs with configurable depth, baud rate divisor logic, and asynchronous oversampling. The VP UART (`uart.cpp`) writes transmitted bytes directly to the host's `stdout` and reads received bytes from `stdin` (or a TCP socket). The baud rate divisor register is accepted but ignored; transmission is instantaneous. This is acceptable because no firmware should depend on UART timing for correctness (only for baud rate configuration, which is a hardware concern).

2.6 Video Pipeline

The RTL video pipeline consists of `fb_ctrl.sv` (framebuffer DMA scanout), `vga_timing.sv` (640×480 timing generator), `pmod_vga_dac.sv` (analog output), and optionally `tmds_encoder.sv` (DVI). The VP replaces this entire chain with a single `SC_THREAD` in `fb_ctrl.cpp` that:

1. Waits for a vsync event (triggered by the ISS writing to the vsync control register).
2. Reads the framebuffer base address and stride from registers.
3. Reads 320×200 bytes (8bpp indexed color) from VP memory via TLM transactions.

4. Applies the PLAYPAL palette (256 entries of 24-bit RGB) to expand to 32-bit RGBA.
5. Scales to 640×480 using nearest-neighbor (2× horizontal, 2.4× vertical with line doubling).
6. Pushes the frame to an SDL2 window for display on the host.

The vsync interrupt is asserted after the frame is pushed, allowing the firmware to swap framebuffers in the double-buffer scheme.

2.7 Audio Pipeline

The RTL audio path uses `i2s_tx.sv` for I2S framing and `audio_dma.sv` for ring-buffer DMA. The VP replaces both with `audio_out.cpp`, which:

1. Monitors the audio DMA ring buffer registers (base address, read pointer, write pointer, buffer size).
2. Periodically (at the configured sample rate, e.g., 11025 Hz for DOOM) reads samples from VP memory.
3. Feeds samples to an SDL2 audio callback for host playback.
4. Asserts an underrun interrupt if the read pointer catches the write pointer.

2.8 DMA Engine

The `dma_engine.cpp` module models the AXI DMA engine (`axi_dma.sv` in RTL). It is a bus master with its own TLM initiator socket bound to the bus. When firmware writes a source address, destination address, and byte count to the DMA registers, the DMA engine performs a burst copy via TLM transactions. The copy is executed as a single `SC_THREAD` invocation: read source data into a local buffer, then write to the destination. An interrupt is asserted on completion.

The DMA engine must invalidate LR/SC reservations if it writes to a reserved address. This is modeled by notifying the ISS via a shared reservation-invalidation callback.

2.9 Debug Architecture

The RTL debug path uses a JTAG TAP controller (`jtag_tap.sv`) and a RISC-V Debug Module (`riscv_dm.sv`). The VP replaces both with a GDB Remote Serial Protocol (RSP) server implemented in `gdb_server.cpp`. This module:

- Opens a TCP listening socket (default port 5005).
- Speaks the GDB RSP text protocol (packets delimited by `$...#xx`).
- Supports commands: `g` (read registers), `G` (write registers), `m` (read memory), `M` (write memory), `s` (single step), `c` (continue), `Z0`/`z0` (set/clear software breakpoints).
- Halts the ISS by clearing a "running" flag checked at the top of the fetch loop; the ISS calls `wait()` until the flag is set again.

To connect: `riscv32-unknown-elf-gdb -ex "target remote :5005" firmware.elf`.

3 Detailed Directory Walkthrough

This section provides multi-paragraph descriptions of every directory and every file in the GamingCPU-VP repository.

3.1 deps/

3.1.1 deps/setup.sh

This shell script automates the acquisition and compilation of all external dependencies required by the GamingCPU-VP. It downloads the Accellera SystemC reference implementation (version 2.3.4 or later) from the official GitHub repository, configures it with CMake, and installs it to a local prefix (`deps/systemc-install/`). The script also verifies that the TLM-2.0 headers (bundled with SystemC) are present. If SDL2 development libraries are not detected via `pkg-config`, the script downloads and builds SDL2 from source. Finally, it checks for the RISC-V GNU toolchain (`riscv32-unknown-elf-gcc`) in `$PATH` and emits a warning if absent.

The script is idempotent: re-running it skips already-completed steps. It exports environment variables `SYSTEMC_HOME` and `SDL2_HOME` for use by the main CMake build. Failure modes include network unavailability (mitigated by checking for cached tarballs) and incompatible host compiler versions (SystemC requires C++17; the script verifies `g++ -std=c++17` support).

3.2 src/main.cpp

This file contains the `sc_main()` function, the entry point for the SystemC simulation. It performs the following sequence:

1. Parses command-line arguments: `-elf` (firmware path), `-sd-image` (SD card image path), `-gdb-port` (GDB server port, 0 to disable), `-quantum` (temporal decoupling quantum in microseconds), `-sdl-video` (enable SDL2 video window), `-sdl-

audio (enable SDL2 audio output), -trace (enable instruction trace to file), -exit-on-tohost (exit when a write to the HTIF tohost address is detected, used for ISA compliance tests).

2. Instantiates the top-level platform module (GamingCPU_VP).
3. Calls the ELF loader to parse the firmware binary and load segments into the memory model, setting the ISS program counter to the ELF entry point.
4. If an SD image path is provided, passes it to the SD controller model.
5. Calls sc_start() to begin simulation.
6. On simulation exit, prints performance statistics: simulated time, wall-clock time, instructions executed, and MIPS.

This file mirrors the RTL's top_arty_a7.sv in that it instantiates and wires together all top-level components. However, whereas the RTL top instantiates clock wizards, reset synchronizers, and MIG DDR controllers, the VP top simply constructs C++ objects and binds TLM sockets.

3.3 src/platform/

3.3.1 src/platform/gamingcpu_vp.h and gamingcpu_vp.cpp

These files define the top-level platform class, GamingCPU_VP, which is an sc_module that owns and interconnects all subsystem instances. It replaces rtl/subsys/soc_axi_top.sv and periph_axi_shell.sv.

The constructor instantiates: one ISS (CPU), one TLM_Bus (interconnect), one Memory (128 MB RAM), one BootROM (64 KB), one CLINT, one PLIC, one UART, one GPIO, one Timer64, one SD_Ctrl, one FB_Ctrl (video), one AudioOut, one DMA_Engine, and one GDB_Server.

After instantiation, it binds the ISS's initiator socket to the bus's target socket, binds the DMA engine's initiator socket to the bus (as a second master), and binds each peripheral's target socket to the bus's initiator sockets. It then configures the bus address map by calling bus.map() for each peripheral with the base address and size from platform_config.h.

Interrupt wiring is performed by connecting sc_signal<bool> signals between peripheral IRQ outputs and the PLIC's interrupt inputs. The CLINT's timer IRQ output and the PLIC's external IRQ output are connected to the ISS's interrupt input signals.

3.3.2 src/platform/platform_config.h

This header defines all address map constants as constexpr values, generated from specs/memory_map.yaml. It also defines IRQ source IDs (used as PLIC input indices), clock frequencies, and default quantum values. This file is the single source of truth for all magic numbers in the VP; no other file should contain hardcoded addresses.

3.4 src/cpu/

3.4.1 src/cpu/iss.h and iss.cpp

The Instruction Set Simulator is the largest and most critical module in the VP. The ISS class inherits from sc_module and contains:

- A 32-element int32_t register file array.
- A uint32_t pc program counter.
- A CSR object (see csr.h).
- An MMU object (see mmu.h).
- A tlm_utils::simple_initiator_socket for memory access.
- A tlm_quantumkeeper for temporal decoupling.
- A sc_in<bool> for timer interrupt.
- A sc_in<bool> for external interrupt.
- Breakpoint data structures for GDB.

The run() SC_THREAD implements the main loop described in Section 2.1. Memory access methods (mem_read8/16/32, mem_write8/16/32) first check for a valid DMI pointer; if present, they dereference it directly. Otherwise, they construct a tlm_generic_payload, set the address, data pointer, command, and data length, and call b_transport on the initiator socket. If the MMU is active, the virtual address is translated before the memory access.

The ISS also provides an instruction trace facility: if enabled, every retired instruction is logged as a line in a trace file with format: PC INSTR_HEX DISASM RD=VAL. This trace is the primary mechanism for cross-validation against RTL (see Section 7.11).

In the RTL, this module's functionality is distributed across rv32_core.sv, fetch.sv, decode.sv, execute.sv, mem_stage.sv, writeback.sv, hazard_unit.sv, and branch_predictor.sv. In the VP, all of these collapse into a single sequential loop because there are no pipeline hazards to resolve, no branch mispredictions to recover from, and no cache miss stalls to handle. On FPGA, re-expanding this into a pipeline is the most complex engineering task in the project.

3.4.2 src/cpu/decode.h and decode.cpp

The decoder is a stateless function `decode(uint32_t instr)` that returns a `DecodedInstr` structure. It handles:

- RV32I base integer instructions (LUI, AUIPC, JAL, JALR, branches, loads, stores, ALU immediate, ALU register).
- RV32M multiplication/division (detected by funct7 = 0x01).
- RV32A atomics (detected by opcode 0x2F).
- RV32C compressed instructions (detected by bits[1:0] != 3). Compressed instructions are expanded to their 32-bit equivalents using a lookup table, then decoded normally.
- System instructions (ECALL, EBREAK, xRET, SFENCE.VMA, WFI, CSR operations).

The decoder does not raise exceptions itself; it marks illegal instructions with a flag in the `DecodedInstr`, and the execute stage raises the `ILLEGAL_INSTRUCTION` exception.

This mirrors `rtl/cpu/core/decode.sv`, which performs the same field extraction in combinational logic. The key difference is that the RTL decoder must produce all control signals in a single clock cycle, while the VP decoder can use arbitrarily complex C++ logic without timing constraints.

3.4.3 src/cpu/execute.h and execute.cpp

The execute module is a method `ISS::execute(DecodedInstr& d)` that performs the operation specified by the decoded instruction. It contains a `switch` over instruction types with approximately 60 cases covering all RV32IMAC operations.

Load and store instructions invoke the ISS's memory access methods, which may trigger MMU translation and then either DMI access or TLM transactions. Branch instructions compute the comparison, set `next_pc` accordingly, and do *not* model branch prediction (there is no penalty for taken branches in the VP).

CSR instructions (`CSRRW`, `CSRRS`, `CSRRC` and their immediate variants) invoke `csr.read()` and `csr.write()` with appropriate read-modify-write semantics.

`ECALL` and `EBREAK` set the exception cause and invoke the trap handler. `MRET`/`SRET` restore privilege level and PC from the appropriate CSRs. `WFI` calls `sc_core::wait()` on an event that is notified when an interrupt arrives, effectively putting the ISS to sleep until the next interrupt.

`FENCE` is a no-op in the VP (there is no cache to flush). `FENCE.I` invalidates the DMI pointer to instruction memory, forcing subsequent fetches through the TLM path until a new DMI pointer is obtained. This models the RTL's I-cache invalidation.

3.4.4 src/cpu/rv32m.h and rv32m.cpp

This module implements the M standard extension. `MUL` computes the lower 32 bits of a 64-bit signed product. `MULH` computes the upper 32 bits of a signed×signed product. `MULHSU` computes the upper 32 bits of a signed×unsigned product. `MULHU` computes the upper 32 bits of an unsigned×unsigned product. `DIV` and `REM` handle division by zero (result is -1 for `DIV`, dividend for `REM`) and signed overflow ($-2^{31} \div -1$, result is -2^{31} for `DIV`, 0 for `REM`).

In the RTL, `rv32m_muldiv.sv` implements a multi-cycle sequential multiplier/divider that stalls the pipeline for 32+ cycles. In the VP, these operations complete in a single C++ statement.

3.4.5 src/cpu/rv32a.h and rv32a.cpp

This module implements the A standard extension (atomic memory operations). Only `LR.W` and `SC.W` are required for RV32IMAC; AMO instructions (AMOSWAP, AMOADD, etc.) may be included for completeness but are not exercised by the DOOM port.

`LR.W` loads a word from memory and records the address in the reservation set. `SC.W` checks if the reservation is still valid for the target address; if so, it stores the word and writes 0 to `rd` (success); otherwise, it writes a nonzero value to `rd` (failure) without performing the store.

The reservation is invalidated by any store (from the ISS or DMA) to an address within the reservation granule (typically 32 bytes, configurable to match the RTL's cache line size).

3.4.6 src/cpu/csr.h and csr.cpp

The CSR file implements all M-mode and S-mode CSRs required by the RV32IMAC privileged specification. The implementation is a `struct` with named fields for each CSR, plus `read()` and `write()` methods that enforce WARL semantics.

Notable behaviors: writing to `mstatus` masks the MPP field to legal values (M or U mode); writing to `satp` triggers a TLB flush; reading `mtime` redirects to the CLINT's mtime value (memory-mapped, but some

implementations also expose it as a CSR); `mcycle` and `minstret` are incremented by the ISS main loop.

3.4.7 src/cpu/mmu.h and mmu.cpp

Described in Section 2.1. The file contains the `MMU` class with methods `translate(vaddr, access_type, privilege)` returning either a physical address or a page fault exception code. The TLB is implemented as a `std::unordered_map<uint32_t, TLBEntry>` keyed by VPN.

Failure modes include: accessing a page with insufficient permissions (e.g., writing to a read-only page), accessing a page marked as non-user from U-mode, superpage misalignment (VPN[0] must be zero for 4 MB superpages), and infinite page walk loops (guarded by a depth counter).

3.4.8 src/cpu/trap.h and trap.cpp

The trap handling module implements `take_trap(cause, tval)` and `return_from_trap(privilege)`. It encapsulates the logic for saving/restoring CSRs during trap entry/exit, including delegation (checking `medeleg`/`mideleg` to determine whether a trap should be handled in M-mode or S-mode).

3.4.9 src/cpu/rv32_defs.h

Header-only file containing opcode constants, CSR addresses, exception cause codes, and instruction format masks. Equivalent to `rtl/cpu/pkg/rv32_pkg.sv`.

3.5 src/bus/

3.5.1 src/bus/tlm_bus.h and tlm_bus.cpp

The TLM bus is the central interconnect. It implements `tlm_target_socket` (for masters) and an array of `tlm_initiator_socket` (for slaves). The `b_transport` implementation performs address decoding, offset adjustment, and forwarding. It also implements `get_direct_mem_ptr` by forwarding DMI requests to the appropriate target and caching the result.

The bus supports two masters: the ISS and the DMA engine. In the VP's loosely-timed model, bus contention is not modeled; both masters can issue transactions concurrently (they are actually serialized by the SystemC cooperative threading model, but this is invisible to the models). In the RTL, the AXI crossbar arbitrates between masters with round-robin or priority-based schemes.

Failure modes: address decode miss (returns TLM error), overlapping address ranges in the map (detected at elaboration with an assertion), and DMI invalidation (if a peripheral is remapped, which does not occur in normal operation).

3.6 src/mem/

3.6.1 src/mem/memory.h and memory.cpp

The `Memory` class models all read/write memory in the system (SRAM and DDR3 unified). Constructor parameters: name, base address, and size. The backing store is `std::vector<uint8_t>` initialized to zero.

The `b_transport` handler extracts the address offset, checks bounds, and performs a `memcpy` between the transaction data pointer and the internal vector. The `get_direct_mem_ptr` handler returns a DMI descriptor with the vector's `data()` pointer, enabling zero-copy access.

In the RTL, this corresponds to `sram_dual_port.sv` (on-chip SRAM), the MIG DDR3 controller (`mig_ddr3.xci` + `mig_axi_wrap.sv`), and the cache hierarchy (`icache.sv`, `dcache.sv`). The VP collapses all of this into a flat array because caches are transparent to software (modulo self-modifying code, handled by `FENCE.I` → DMI invalidation).

3.6.2 src/mem/bootrom.h and bootrom.cpp

The `BootROM` class is identical to `Memory` except that `b_transport` rejects write commands with `TLM_COMMAND_ERROR_RESPONSE`. It is initialized by reading a binary file (the BootROM image) at elaboration. DMI is granted with read-only permission.

3.7 src/irq/

3.7.1 src/irq/clint.h and clint.cpp

The CLINT module contains an `SC_THREAD` (`tick_thread`) that increments `mtime` at the configured tick interval. The thread uses `sc_core::wait(tick_period)` to advance simulated time between increments. After each increment, it compares `mtime` with `mtimecmp` and drives the *timer_irq* output accordingly.

Register map (matching `specs/registers/clint.yaml`):

Offset	Name	Access
0x0000	msip	RW
0x4000	mtimecmp_lo	RW
0x4004	mtimecmp_hi	RW
0xBFF8	mtime_lo	RW
0xBFFC	mtime_hi	RW

3.7.2 src/irq/plic.h and plic.cpp

The PLIC module implements priority registers (one per interrupt source), pending bits, enable bits (one bit per source per context), a priority threshold, and claim/complete registers. It is purely register-driven with no internal threads; interrupt evaluation is triggered by writes to the pending bits (from peripherals) or enable bits (from software).

The RTL PLIC (`plic.sv`) uses combinational priority encoding. The VP uses a simple loop over all sources to find the highest-priority pending-and-enabled interrupt.

3.8 src/io/

3.8.1 src/io/uart.h and uart.cpp

The UART implements a 16550-compatible register interface. The TX register write handler calls `putchar()` (or writes to a TCP socket if configured). The RX path uses a background `SC_THREAD` that polls `stdin` (via non-blocking `read()`) and fills an internal 16-byte FIFO. The Line Status Register (LSR) reflects FIFO empty/full state. The UART asserts its IRQ output when enabled interrupt conditions are met (TX empty, RX data available, etc.).

On FPGA, the UART (`uart16550.sv`) handles baud rate generation, start/stop bit framing, and oversampling. None of this is modeled in the VP.

3.8.2 src/io/gpio.h and gpio.cpp

The GPIO module implements direction, output data, input data, and interrupt mask/status registers. The "input" register can be driven by a host-side callback (e.g., SDL2 keyboard events mapped to GPIO pins for DOOM input). The GPIO asserts an interrupt when a configured input pin transitions and the corresponding interrupt mask bit is set.

3.8.3 src/io/timer.h and timer.cpp

A 64-bit system timer with a compare register and IRQ. Similar to the CLINT's mtime but exposed as a peripheral on the MMIO bus (at a different address) for use by userspace or S-mode code.

3.8.4 src/io/spi.h and spi.cpp

A minimal SPI master model. In the VP, SPI transactions are modeled as register writes (command, data) and reads (received data). The SPI clock rate, polarity, and phase registers are accepted but not used for timing. The SPI master is primarily used by the SD controller in SPI mode.

3.9 src/sd/

3.9.1 src/sd/sd_ctrl.h and sd_ctrl.cpp

The SD controller replaces `sd_spi_ctrl.sv` and `sd_sdio_ctrl.sv`. Register interface: command register (CMD0, CMD17 single-block read, CMD18 multi-block read, etc.), argument register (block address), data address register (DMA destination in memory), burst length register, and status register (busy, error, done, IRQ).

When firmware writes to the command register with CMD17 and a block address, the SD controller:

1. Opens the backing file (if not already open).
2. Seeks to offset `block_addr * 512`.
3. Reads 512 bytes into an internal buffer.
4. Writes the buffer to VP memory at the data address register value via a TLM write transaction on its initiator socket.
5. Sets the done flag and asserts the IRQ.

CMD18 (multi-block) repeats this for `burst_length` consecutive blocks, incrementing both the block address and the destination pointer.

3.9.2 src/sd/sd_card_model.h and sd_card_model.cpp

The virtual SD card backend. It holds an `std::ifstream` to the host-side `.img` file and provides `read_block(addr, buf)` and `write_block(addr, buf)` methods. Write support is optional (DOOM only reads from the WAD). The image file is expected to contain a FAT32 filesystem with `/doom/DOOM1.WAD` at the expected location, created by `packaging/sd/make_sd_image.sh`.

3.10 src/video/

3.10.1 src/video/fb_ctrl.h and fb_ctrl.cpp

Described in Section 2.6. Key registers: FB0_ADDR (framebuffer 0 base), FB1_ADDR (framebuffer 1 base), STRIDE (bytes per scanline), PAL_ADDR (palette base in memory), VSYNC_CTRL (swap buffer / enable scanout), VSYNC_STATUS (current buffer, vsync pending).

The SC_THREAD (scanout_thread) waits on an sc_event triggered by a write to VSYNC_CTRL. It then reads the framebuffer, applies the palette, and calls SDL2's SDL_UpdateTexture and SDL_RenderPresent. If SDL2 is disabled (headless mode), the thread computes a CRC32 checksum of the frame and logs it, enabling golden-frame validation without a display.

3.10.2 src/video/palette.h and palette.cpp

Utility functions for loading PLAYPAL and COLORMAP data from memory. PLAYPAL is 256 entries $\times$ 3 bytes (RGB) $\times$ 14 palettes. COLORMAP is 256 entries $\times$ 34 maps (brightness levels). These are loaded by DOOM into memory from the WAD file; the VP reads them from the memory model at palette upload time.

3.11 src/audio/

3.11.1 src/audio/audio_out.h and audio_out.cpp

Described in Section 2.7. Key registers: RING_BASE (ring buffer base address), RING_SIZE (buffer size in bytes), RD_PTR (hardware read pointer), WR_PTR (software write pointer), SAMPLE_RATE (divisor), CTRL (enable, format), STATUS (underrun flag, IRQ pending).

The SC_THREAD (playback_thread) runs a loop that: (1) computes the number of available samples (WR_PTR - RD_PTR modulo RING_SIZE), (2) reads up to one SDL2 audio buffer's worth of samples from VP memory, (3) queues them for SDL2 audio callback playback, (4) advances RD_PTR, (5) waits for the sample period. If available samples drop to zero, the underrun flag is set and the IRQ is asserted.

3.12 src/dma/

3.12.1 src/dma/dma_engine.h and dma_engine.cpp

The DMA engine models rtl/bus/axi/axi_dma.sv and rtl/subsys/dma_subsys.sv. It has both a target socket (register access from the CPU) and an initiator socket (memory access as a bus master).

Registers: SRC_ADDR, DST_ADDR, BYTE_COUNT, CTRL (start, IRQ enable), STATUS (busy, done, error).

When firmware writes to CTRL with the start bit set, the DMA SC_THREAD:

1. Reads BYTE_COUNT bytes from SRC_ADDR via TLM read transactions on the initiator socket.
2. Writes the data to DST_ADDR via TLM write transactions.
3. Checks if DST_ADDR overlaps the ISS's LR/SC reservation set; if so, invalidates it.
4. Sets the done flag, clears the busy flag, and asserts the IRQ.

The DMA performs the transfer in chunks (configurable burst size, default 256 bytes) to allow interleaving with ISS activity at quantum boundaries.

3.13 src/debug/

3.13.1 src/debug/gdb_server.h and gdb_server.cpp

Described in Section 2.9. The GDB server is instantiated by the platform only if -gdb-port is nonzero. It holds a reference to the ISS for register/memory access and breakpoint management.

The implementation uses POSIX sockets (socket(), bind(), listen(), accept()) in a background SC_THREAD that blocks on accept() until a GDB client connects. Once connected, it enters a command loop: read a packet, parse the command, perform the action (read/write regs, read/write memory, set breakpoint, single step, continue), send the response.

Software breakpoints are implemented by replacing the instruction at the breakpoint address with EBREAK (0x00100073) and storing the original instruction. When the ISS hits EBREAK and a breakpoint is registered at that PC, the ISS halts and notifies the GDB server, which sends a stop reply to the client.

3.14 src/util/

3.14.1 src/util/elf_loader.h and elf_loader.cpp

The ELF loader parses 32-bit ELF files (little-endian, RISC-V machine type). It iterates over program headers of type PT_LOAD, reads the segment data from the file, and writes it into the VP memory model via direct pointer access (using the DMI pointer for RAM regions). It

extracts the entry point address from the ELF header and returns it to `sc_main()` for ISS initialization.

The loader validates: ELF magic number, 32-bit class, little-endian encoding, RISC-V machine type (0xF3), and segment address ranges (ensuring they fall within the VP's memory map). Invalid ELF files produce fatal errors.

3.14.2 `src/util/logging.h` and `logging.cpp`

Wrapper around SystemC's `sc_report` mechanism. Provides macros: `VP_INFO`, `VP_WARN`, `VP_ERROR`, `VP_FATAL`. Also provides a transaction trace facility: when enabled, every TLM transaction on the bus is logged with timestamp, address, data, and read/write flag. This trace can be compared with RTL AXI transaction logs for cross-validation.

3.15 `sw/`

The entire `sw/` directory is *reused verbatim* from the RTL repository. No modifications are needed because the VP implements the identical register map and memory map. The cross-compiled ELF binaries produced by the RISC-V toolchain execute on the VP without changes.

4 Memory Map Specification

Table 2 shows the complete system memory map.

Table 2: System Memory Map

Base	End	Size	Peripheral
0x0000_0000	0x0000_FFFF	64 KB	BootROM
0x0100_0000	0x0100_FFFF	64 KB	On-chip SRAM
0x0200_0000	0x0200_FFFF	64 KB	CLINT
0x0C00_0000	0x0FFF_FFFF	64 MB	PLIC
0x1000_0000	0x1000_0FFF	4 KB	UART
0x1000_1000	0x1000_1FFF	4 KB	GPIO
0x1000_2000	0x1000_2FFF	4 KB	Timer64
0x1000_3000	0x1000_3FFF	4 KB	SPI Master
0x1000_4000	0x1000_4FFF	4 KB	SD Controller
0x1000_5000	0x1000_5FFF	4 KB	DMA Engine
0x1000_6000	0x1000_6FFF	4 KB	Video (FB Ctrl)
0x1000_7000	0x1000_7FFF	4 KB	Audio (I2S/PWM)
0x8000_0000	0x87FF_FFFF	128 MB	DDR3 RAM

4.1 Interrupt Sources

Table 3: PLIC Interrupt Source Assignment

IRQ ID	Source	Default Priority
1	UART RX/TX	3
2	GPIO edge	2
3	Timer64 compare	4
4	SD controller done	5
5	DMA complete	5
6	Video vsync	6
7	Audio underrun	7

4.2 Framebuffer Layout

DOOM renders at 320×200 pixels, 8 bits per pixel (indexed color). The framebuffer occupies 64 000 bytes. Two framebuffers are allocated in DDR for double buffering:

- FB0: `0x8400_0000` – `0x8400_F9FF` (64 000 bytes)
- FB1: `0x8401_0000` – `0x8401_F9FF` (64 000 bytes)
- Palette (PLAYPAL): `0x8402_0000` (768 bytes per palette × 14)
- Colormap: `0x8403_0000` (256 bytes × 34 maps)

4.3 Audio Ring Buffer Layout

The audio ring buffer is allocated in DDR at `0x8404_0000` with a default size of 4 KB. DOOM generates 11025 Hz mono 16-bit samples (approximately 22 KB/s). At 4 KB, the buffer holds approximately 180 ms of audio, providing sufficient margin for a 28 ms frame period (35 Hz).

5 Transaction Flow Scenarios

5.1 CPU Instruction Fetch

```
ISS                     BUS               MEMORY
 |                       |                  |
 |--[1] DMI check------->|                  |
 |  (hit: direct ptr)    |                  |
 |<--instr word----------|--(ptr deref)---|
 |                       |                  |
 |  (miss: TLM path)     |                  |
 |--[2] b_transport------>|                  |
 |  addr=PC, cmd=READ    |--[3] forward--->|
 |                       |  addr=PC-base   |
 |                       |<--[4] data------|
 |<--[5] instr word------|                  |
 |                       |                  |
 |--[6] DMI request------>|--[7] forward-->|
 |<--[8] DMI ptr---------|<--DMI desc------|
 |  (cache for future)   |                  |
```

Listing 2: Instruction fetch transaction flow

Step [1] is the fast path: if a DMI pointer is cached for the address range containing PC, the instruction word is

read directly from the host memory pointer. Steps [2]-[8] occur only on the first access to a memory region or after a DMI invalidation (`FENCE.I`).

5.2 MMU Page Walk

```
ISS          MMU          BUS         MEMORY
 |            |            |            |
 |--vaddr---->|            |            |
 |            |--TLB lookup|            |
 |            |  (miss)    |            |
 |            |            |            |
 |            |--L1 PTE read----------->|
 |            |  addr=satp.PPN*4K       |
 |            |       +VPN1*4           |
 |            |<-------PTE1-------------|
 |            |            |            |
 |            |  (pointer, not leaf)    |
 |            |--L2 PTE read----------->|
 |            |  addr=PTE1.PPN*4K       |
 |            |       +VPN0*4           |
 |            |<-------PTE2------------|
 |            |            |            |
 |            |  (leaf: check perms)   |
 |            |--TLB insert|            |
 |<--paddr----|            |            |
```

Listing 3: Sv32 page walk transaction flow

5.3 UART Transmit

```
ISS         BUS         UART        HOST
 |           |            |           |
 |--store--->|            |           |
 | addr=UART+TX_REG      |           |
 | data=0x41 ('A')       |           |
 |           |--forward-->|           |
 |           |            |--putchar->|
 |           |            |  (stdout) |
 |           |<--OK-------|           |
 |<--OK------|            |           |
```

Listing 4: UART TX transaction flow

5.4 Timer Interrupt

```
CLINT                ISS
 |                    |
 |  mtime++           |
 |  mtime>=mtimecmp?  |
 |  YES               |
 |--timer_irq=1------>|
 |                    |  (at quantum boundary)
 |                    |  check_pending_interrupts()
 |                    |  mip.MTIP && mie.MTIE
 |                    |  && mstatus.MIE
 |                    |  -> take_trap(M_TIMER_IRQ)
 |                    |  save pc->mepc
 |                    |  pc = mtvec
 |                    |  ... execute ISR ...
 |                    |  write mtimecmp (new val)
 |                    |
 |  (mtimecmp updated)|
 |  mtime<mtimecmp    |
 |--timer_irq=0------>|
```

Listing 5: CLINT timer interrupt flow

5.5 DMA Framebuffer Transfer

```
ISS         DMA_ENGINE       BUS        MEMORY
 |            |               |           |
 |--cfg regs:|               |           |
 | SRC=doom_fb               |           |
 | DST=FB0_ADDR              |           |
 | LEN=64000                 |           |
 | CTRL=START                |           |
 |            |               |           |
 |            |--read 256B-->|---------->|
 |            |<--data-------|<----------|
 |            |--write 256B->|---------->|
 |            |<--OK---------|<----------|
 |            | (repeat x250)|           |
 |            |               |           |
 |            |--IRQ assert->|           |
 |<----------(PLIC)---------|           |
```

Listing 6: DMA frame transfer flow

5.6 DOOM Frame Render Path

```
DOOM SW     ISS    SD_CTRL   MEM    FB_CTRL  SDL2
  |          |       |        |        |        |
  |--read WAD lump----------->|        |        |
  |          |        |  (cached in RAM)        |
  |          |       |        |        |        |
  |--BSP traverse, render columns     |        |
  |  (fixed-point math)       |        |        |
  |--write pixels to back FB->|        |        |
  |          |       |        |        |        |
  |--write VSYNC_CTRL-------->|        |        |
  |          |       |        |--evt->|        |
  |          |       |        |       |--read  |
  |          |       |        |       |  FB--->|
  |          |       |        |       |--apply |
  |          |       |        |       | palette|
  |          |       |        |       |--scale |
  |          |       |        |       |--render|
  |          |       |        |       |------->|
  |          |       |        |       |  (SDL2 |
  |          |       |        |       |display)|
  |          |       |        |       |        |
  |          |       |        |  vsync IRQ     |
  |<-------(PLIC)-------------|        |        |
  |--swap buffer ptr          |        |        |
  |--push audio samples------>|        |        |
```

Listing 7: Complete DOOM frame path

6 DOOM Integration Analysis

6.1 Why DOOM Works on This Architecture

The original DOOM engine (id Software, 1993) was designed for 32-bit x86 processors running at 33-66 MHz with 4-8 MB of RAM. The GamingCPU SoC provides a 32-bit RISC-V core at 100 MHz with 128 MB of DDR3 RAM, which is substantially more capable than the original target. DOOM's software renderer uses a BSP (Binary Space Partitioning) algorithm with column-by-column drawing, requiring only integer arithmetic (no floating-point unit), which aligns perfectly with the RV32IMAC ISA.

6.2 Required ISA Extensions

- **I (Base Integer):** All control flow, memory access, and ALU operations.
- **M (Multiplication/Division):** Fixed-point math for rendering: wall height calculations, texture mapping, lighting. Without M, these would require software multiply routines, reducing performance by approximately 10× for multiply-heavy loops.

- **A (Atomics):** Required only if running under an OS with potential concurrency; not needed for bare-metal DOOM. Included for OS boot path compatibility.
- **C (Compressed):** Reduces code size by approximately 25-30%, which improves instruction cache hit rate in the RTL and reduces memory bandwidth in the VP.

6.3 Fixed-Point Math Considerations

DOOM uses 16.16 fixed-point arithmetic extensively: angles, sine/cosine tables, wall distances, texture offsets. The critical operation is `FixedMul: (a * b) » 16`, which requires a 64-bit intermediate product. With the M extension, this compiles to `MUL` (lower 32 bits) + `MULH` (upper 32 bits), followed by a shift and combine. Without M, GCC emits a software multiply routine consuming approximately 100 cycles per call.

6.4 Framebuffer Constraints

DOOM renders at 320×200 at 8bpp (indexed color). The 64 KB framebuffer fits comfortably in DDR. The palette (PLAYPAL) provides 14 palettes of 256 RGB entries; palette switching (e.g., red flash on damage) is achieved by changing the palette pointer register, not by modifying pixel data. The VP's framebuffer controller must handle this palette indirection correctly.

6.5 Audio Timing

DOOM generates audio at 11025 Hz (or 22050 Hz in some ports), mono 16-bit. The audio DMA engine must consume samples at this rate without underrun. At 35 Hz frame rate, each frame generates approximately 11025/35 = 315 samples (630 bytes). The 4 KB ring buffer provides approximately 6 frames of headroom.

6.6 SD Card Access Patterns

DOOM loads WAD lumps on demand. The initial load reads the WAD directory (a few KB at the end of the file), then the level's geometry, textures, and sound data (typically 100-500 KB per level). During gameplay, lump reads are infrequent (new sound effects, occasional texture loads). The SD controller must support multi-block reads with DMA to achieve acceptable load times; single-block SPI reads at 400 kHz would make level loading painfully slow.

6.7 Expected Performance Bottlenecks

On the VP, performance is dominated by the ISS interpretation overhead. Each RISC-V instruction requires approximately 20-100 host x86 instructions to decode and execute. Expected VP performance is 1-5 MIPS, which means DOOM at 35 Hz will run at approximately 1-3× real-time on a modern host CPU. This is sufficient for functional validation but not for interactive play in all cases.

On the RTL (FPGA), the bottleneck shifts to memory bandwidth: the framebuffer scanout DMA and audio DMA compete with CPU instruction fetches and data accesses on the AXI bus.

7 Verification & Testing Strategy

7.1 Unit Testing Philosophy

Every VP module is tested in isolation before integration. Each unit test instantiates the module under test, binds a test harness to its TLM sockets, and injects transactions. Assertions verify expected register values, output signals, and side effects. Unit tests are written in C++ using a minimal assertion framework and are compiled as standalone SystemC executables.

7.2 TLM-Level Assertions

All TLM transactions include response status checking. The bus asserts `SC_ERROR` on address decode failures. Peripherals assert `SC_ERROR` on out-of-range register accesses. Memory asserts `SC_ERROR` on out-of-bounds accesses. These assertions are checked by the test harness and cause immediate test failure.

7.3 Transaction Tracing Strategy

When enabled via `-trace`, the VP logs every TLM transaction to a structured trace file with fields: timestamp (ns), initiator (CPU or DMA), address, data, command (R/W), response, and latency. This trace can be compared with RTL AXI signal traces using a custom diffing tool.

7.4 Deterministic Replay

The VP is fully deterministic: given the same ELF binary, the same SD image, and the same input sequence, it produces identical instruction traces and memory states on

every run. This enables deterministic replay for debugging: capture the instruction trace on a failing run, then replay with breakpoints at specific instruction counts.

7.5 ISA Compliance Flow

The official RISC-V ISA test suite (`riscv-tests`) is executed on the VP as follows:

1. Each test ELF is loaded into the VP.
2. The VP runs until a write to the "tohost" address (typically `0x80001000`) is detected.
3. If `tohost` value is 1, the test passed; any other value indicates failure (the value encodes the failing test case number).
4. Tests: `rv32ui-p-*` (47 tests), `rv32um-p-*` (8 tests), `rv32ua-p-*` (8 tests), plus custom MMU tests (`sv32_page_faults.S`, `tlb_stress.S`).

7.6 Memory Corruption Detection

The memory model can optionally enable guard pages: 4 KB regions at the boundaries of each memory region filled with a poison pattern (`0xDEADBEEF`). Any access to a guard page indicates a buffer overrun or stack overflow and triggers an immediate `SC_FATAL`.

7.7 Interrupt Stress Tests

Dedicated test programs configure the CLINT and PLIC to generate interrupts at maximum rate, verifying that: (a) no interrupts are lost, (b) interrupt priorities are respected, (c) claim/complete handshakes work correctly, and (d) the ISS correctly saves/restores context across nested traps (if delegation is configured).

7.8 Bus Contention Tests

While the VP does not model true bus contention (transactions are serialized by SystemC's cooperative scheduler), contention test programs configure the DMA engine to perform large transfers simultaneously with CPU-intensive computation. The test verifies that DMA transfers complete correctly and that CPU memory accesses are not corrupted.

7.9 SDL Rendering Validation

In headless mode (no SDL2 window), the VP computes a CRC32 checksum of each rendered frame. A golden checksum file (generated from a known-good run) is compared against. Any checksum mismatch indicates a rendering bug. This enables automated visual regression testing in CI.

7.10 Golden Frame Checksum Strategy

1. Run DOOM on the VP for N frames with a scripted input sequence (recorded key events with timestamps).
2. Capture CRC32 checksums of all N frames.
3. Store as `test/golden_frames.txt`.
4. In CI, re-run with the same input and compare checksums.
5. Any mismatch triggers a failure and dumps the frame as PNG for visual inspection.

7.11 Cross-Validation Against RTL

The VP and the Verilator-simulated RTL execute the same firmware binary. Both produce instruction-retirement traces with format:

```
CYCLE PC INSTR_HEX RD_ADDR RD_DATA
```

A diffing script compares traces line-by-line, ignoring the cycle count (which will differ due to pipeline effects) and comparing only PC, instruction, and register write-back values. The first divergence point identifies the bug. Common divergence causes:

- ISS decode error (wrong immediate sign extension).
- RTL pipeline hazard forwarding bug.
- MMU permission check mismatch.
- CSR WARL masking difference.
- Interrupt timing (VP delivers at quantum boundary; RTL delivers at pipeline commit).

7.12 Continuous Integration Structure

```
Stage 1: Build
  - Build SystemC + VP (CMake, Release)
  - Build RISC-V firmware (cross-compile)

Stage 2: Unit Tests (parallel)
  - test_memory, test_bus_routing
  - test_clint, test_plic
  - test_uart_loopback

Stage 3: ISA Compliance (parallel)
  - rv32ui-p-* (47 tests)
  - rv32um-p-* (8 tests)
  - rv32ua-p-* (8 tests)
  - sv32_page_faults, tlb_stress

Stage 4: Integration Tests
  - hello_uart (check stdout)
  - video_color_bars (checksum)
  - audio_sine (checksum)

Stage 5: DOOM Smoke Test
  - Boot DOOM, run 100 frames
  - Compare golden checksums
  - Report MIPS performance
```

Listing 8: CI pipeline stages

8 Build & Execution Instructions

8.1 Dependency Setup

```
# Ubuntu 22.04+
sudo apt install build-essential cmake \
  libsdl2-dev pkg-config

# SystemC (via deps script)
cd GamingCPU-vp && bash deps/setup.sh

# RISC-V Toolchain
# Option A: apt (if available)
sudo apt install gcc-riscv64-unknown-elf
# Option B: build from source
bash sw/toolchain/build_rv32.sh
```

Listing 9: Dependency installation

8.2 CMake Build Flow

```
mkdir build && cd build
cmake .. \
  -DSYSTEMC_HOME=../deps/systemc-install \
  -DCMAKE_BUILD_TYPE=Release
make -j$(nproc)
# Produces: build/gamingcpu-vp
```

Listing 10: Building the VP

8.3 ELF Loading

The `-elf` argument accepts any RV32IMAC ELF binary. The loader reads `PT_LOAD` segments and copies them into the VP memory at their specified virtual addresses. If the MMU is disabled (bare-metal mode), virtual addresses equal physical addresses. The ISS PC is set to the ELF entry point.

8.4 Running hello_uart

```
cd sw/tests/hello_uart && make
cd ../../../build
./gamingcpu-vp --elf ../sw/tests/hello_uart/hello.
    elf
# Expected output:
# [VP] SystemC 2.3.4, TLM 2.0.6
# [VP] Loading ELF: hello.elf (entry=0x80000000)
# [UART] Hello from GamingCPU!
# [UART] UART echo test: type characters...
```

Listing 11: Running hello_uart

8.5 Running DOOM

```
# Build DOOM firmware
cd sw/apps/doom && make
# Create SD image
bash packaging/sd/make_sd_image.sh
# Run
cd build
./gamingcpu-vp \
  --elf ../sw/apps/doom/doom.elf \
  --sd-image ../images/doom1.img \
  --sdl-video --sdl-audio
# An SDL2 window opens with DOOM rendering
```

Listing 12: Running DOOM

8.6 Attaching riscv-gdb

```
# Terminal 1: start VP in GDB-wait mode
./gamingcpu-vp --elf doom.elf \
  --sd-image doom1.img --gdb-port 5005

# Terminal 2: connect GDB
riscv32-unknown-elf-gdb doom.elf \
  -ex "target remote :5005" \
  -ex "break main" \
  -ex "continue"
```

Listing 13: GDB debugging session

8.7 Debugging Checklist

1. **VP crashes immediately:** Check ELF entry point. Verify it falls within the memory map. Check that BootROM or direct-boot mode is configured correctly.
2. **No UART output:** Verify UART base address. Check that the firmware initializes the baud rate divisor (even though the VP ignores it, some firmware checks the status register).
3. **Interrupts not firing:** Verify CLINT/PLIC address map. Check `mie`, `mstatus`.MIE, and PLIC enable registers.
4. **SD read fails:** Verify the `-sd-image` path. Check that the image is a valid FAT32 filesystem.
5. **DOOM renders garbage:** Check palette upload. Verify framebuffer address alignment. Inspect the palette register.
6. **Audio stutters:** Increase ring buffer size. Check sample rate register. Verify DMA completion interrupt is handled.

9 Failure Mode Analysis

9.1 TLM Deadlocks

In a loosely-timed model, deadlocks can occur if two `SC_THREAD`s wait on each other's events without yielding. In the VP, this can happen if the ISS executes `WFI` (wait-for-interrupt) while the CLINT's tick thread is blocked waiting for the SystemC scheduler. Mitigation: the `WFI` implementation uses a timeout

(`wait(quantum, wfi_event)`) so that the ISS periodically re-checks for pending interrupts even if the event is not notified.

9.2 DMI Invalidation Issues

If a peripheral is dynamically remapped (which does not occur in the GamingCPU, but could in future extensions), cached DMI pointers become stale. The VP must call `invalidate_direct_mem_ptr()` on all initiator sockets to force re-acquisition. Failure to invalidate causes silent data corruption: the ISS reads stale data from the old memory region.

`FENCE.I` must invalidate the instruction DMI pointer to ensure that self-modifying code (or code loaded dynamically by the FSBL) is fetched correctly. If `FENCE.I` is not handled, the ISS continues executing stale instructions from the DMI pointer.

9.3 Incorrect MMU Translation

Common bugs: (1) VPN field extraction uses wrong bit positions. (2) Superpage alignment check is missing (VPN[0] must be zero). (3) PTE access/dirty bit updates are not written back to memory. (4) `SFENCE.VMA` does not flush the software TLB. These bugs manifest as page faults on valid accesses or, worse, silent writes to wrong physical addresses.

9.4 Interrupt Starvation

If the ISS quantum is too large, interrupts are delivered infrequently. This can cause: (a) UART FIFO overflow (characters lost), (b) timer ISR missing deadlines, (c) audio underruns due to delayed DMA completion handling. Mitigation: reduce the quantum (default 100 μs; for audio-sensitive workloads, 10 μs may be needed).

9.5 Frame Tearing

Frame tearing occurs if the video scanout thread reads the framebuffer while DOOM is writing to it. In the VP, this is prevented by double buffering: DOOM writes to the back buffer and only signals a swap via `VSYNC_CTRL` when the frame is complete. The scanout thread reads the front buffer. If double buffering is not used (or if the swap signal is missed), tearing manifests as horizontal artifacts.

9.6 Audio Underruns

If DOOM's audio mixing thread cannot fill the ring buffer fast enough (due to ISS slowness or a large quantum), the audio DMA read pointer overtakes the write pointer. The audio output module detects this, outputs silence, and asserts the underrun IRQ. Persistent underruns indicate a performance problem; reducing the quantum or optimizing the audio mixing code may be necessary.

9.7 SD Corruption Scenarios

If the firmware issues an SD read to an out-of-bounds block address (beyond the image file size), the SD controller model detects this and sets the error flag in the status register. If the firmware does not check the error flag, it proceeds with stale or zero data, causing WAD parsing errors. The VP logs a warning in this case.

10 Future Work

10.1 Cycle-Accurate Refinement

The current LT model can be refined to an *approximately-timed* (AT) model by adding pipeline stage delays: instruction fetch (1 cycle + cache miss penalty), decode (1 cycle), execute (1 cycle, variable for M-extension), memory (1 cycle + cache miss), writeback (1 cycle). This enables more accurate performance estimation at the cost of simulation speed (approximately 10× slower).

10.2 Hybrid Co-Simulation with RTL

Using the Verilator-generated C++ model of the RTL, the VP can be extended to co-simulate: the VP provides the CPU ISS and bus model, while individual peripherals are replaced with their Verilated RTL counterparts. This enables verification of specific RTL modules against the VP's golden reference at near-VP speed for the rest of the system.

10.3 Formal Verification Hooks

The ISS can be instrumented to export symbolic execution traces compatible with the RISC-V Formal Interface (RVFI). Each retired instruction emits: PC, instruction, source register values, destination register value, memory address, and memory data. These traces can be consumed by formal verification tools (e.g., riscv-formal) to prove that the ISS correctly implements the ISA specification.

10.4 Trace Export for Waveform Comparison

The VP's transaction trace can be exported in VCD (Value Change Dump) format, allowing visualization in standard waveform viewers (GTKWave). Each TLM transaction is mapped to signal changes on virtual AXI bus signals (ARADDR, ARVALID, RDATA, RVALID, etc.), enabling side-by-side comparison with RTL simulation waveforms. This requires a TLM-to-VCD adapter module that converts blocking transport calls to signal-level events.

11 Conclusion

The GamingCPU-VP provides a complete, production-quality behavioral model of the GamingCPU-SoC custom RISC-V SoC. It executes identical firmware binaries, implements the identical memory map and register interfaces, and serves as the golden reference for RTL verification. The VP enables early software development (including full DOOM integration testing), rapid architectural exploration, and systematic cross-validation against the RTL implementation. Every module in the VP has a direct counterpart in the RTL, ensuring that discrepancies between the two are detectable and actionable. The testing strategy spans unit tests through full-system DOOM smoke tests, with deterministic replay and golden-frame checksumming for automated regression. This document serves as the canonical reference for all engineering decisions in the VP and should be maintained in lockstep with the RTL specification.